SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

NAME: B KIRTHANAA

REGISTER NUMBER: 192324244

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Data Interpretation

Question 1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions: 1. Identify any incorrect or invalid fields. 2. Analyze the issue with Content-Length and its impact. 3. Determine whether the status code matches the response correctness. 4. Suggest corrections.

SOLUTION:

1. Identify any incorrect or invalid fields in the header

Each header line is checked against the HTTP/1.1 specification. Two fields contain errors, while the remaining fields are valid.

Field	Status	Observation
Status line	Valid format	HTTP/1.1 200 OK is correctly formed, but the code may not match the actual response (see point 3).
Date	Incorrect	The format is valid, but 01 April 2026 falls on a Wednesday, not Monday. The weekday and the date contradict each other.
Content-Type	Valid	text/html is a valid media type. Adding a charset is recommended.
Content-Length	Invalid	The value -150 is negative. The field must be a non-negative integer.
Connection	Valid	keep-alive is allowed, but it is the default in HTTP/1.1, so it is optional.

2. Analyze the issue with Content-Length and explain its impact

Content-Length tells the client how many bytes of message body to read. A byte count cannot be negative, so -150 is meaningless and the server has given the client no reliable way to know where the body ends.

Impact:

- Browsers or proxies may reject the response as malformed and show an error instead of the page.
- The client may wait for data that never arrives, causing a hang or timeout.
- With Connection: keep-alive, the connection is reused. A wrong body length makes the next response start at the wrong position, which corrupts later responses on the same connection.
- Incorrect message framing is also a known cause of request/response smuggling attacks, so it is a security risk, not only a display problem.

3. Determine whether the status code matches the response correctness

The status code 200 OK says the request succeeded and a valid response body follows. In this response the message itself is malformed because of the invalid Content-Length, so the status does not truthfully describe the result. The status line is syntactically valid but semantically inconsistent. If the server cannot produce a valid response, the correct behaviour is to return a server-error code such as 500 Internal Server Error. If the content is generated correctly, the header must be fixed so that 200 OK is genuine.

4. Suggest corrections for the identified errors

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

- **Content-Length:** set to the exact size of the body in bytes (150 in this example). If the size is not known in advance, remove it and use Transfer-Encoding: chunked.
- **Date:** use the correct weekday (Wed) or generate the value automatically from the server clock.
- **Content-Type:** add charset=UTF-8 so the browser decodes the text correctly.

Conclusion: The header contains two errors, an invalid negative Content-Length and a wrong weekday in Date. The Content-Length error is serious because it breaks message framing and makes the 200 OK status unreliable. Correct, server-generated headers are essential for reliable content delivery.

Question 2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions: 1. Identify missing or improperly closed elements. 2. Analyze how the browser interprets the structure. 3. Determine the impact on rendering and layout. 4. Suggest corrections.

SOLUTION:

1. Identify missing or improperly closed elements

- **Missing closing tag:** is opened but never closed with .

- **Missing DOCTYPE:** <!DOCTYPE html> is absent, so the browser cannot be told to use standards mode.
- **Missing language attribute:** <html> has no lang attribute.
- **Missing metadata:** there is no <meta charset="UTF-8"> and no viewport meta tag in <head>.
- **Non-semantic structure:** a generic is used where <main> or <section> would give meaning.

2. Analyze how the browser will interpret this structure

Browsers use the HTML parsing algorithm, which is designed to recover from errors instead of failing. When the parser reaches while a <p> is still open, it applies the "implied end tag" rule. It automatically closes the <p> before closing the . The resulting DOM tree is:

```
html
 ├── head
 │    └── title  ("Test Page")
 └── body
      └── div
           └── p  ("Hello World")
```

So the tree is repaired correctly in this case, but the repair is done by the browser, not by the author. Because there is no DOCTYPE, the browser also switches to quirks mode, which uses older, non-standard box model and layout rules.

3. Determine the impact on rendering and layout

- Text "Hello World" is displayed inside a paragraph with the default top and bottom margins, so the visible output looks normal.
- Quirks mode can change the box model, line height, table and image sizing, so the same CSS may look different from a standards-mode page and across browsers.
- Missing charset can cause special characters to display incorrectly.
- Missing viewport settings make the page appear zoomed out on mobile devices.
- If more content were added after the unclosed <p>, the implied closing might happen in an unexpected place and change the layout.

4. Suggest corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

Conclusion: The only improperly closed element is <p>, which the browser silently repairs, but the missing DOCTYPE and metadata still cause quirks-mode rendering risks. Explicitly closing every element and adding the standard document skeleton gives a valid, predictable DOM.

Question 3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions: 1. Interpret the differences in data transmission. 2. Analyze which method is more suitable for login. 3. Identify security risks of GET for authentication. 4. Suggest best practices for secure login.

SOLUTION:

1. Interpret the differences in data transmission between GET and POST

With GET, the form values are appended to the URL as a query string, so the request line itself carries the username and password. With POST, the values are placed in the request body and the URL contains only the path.

```
GET /login?user=admin&pass=1234 HTTP/1.1
Host: example.com

POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 26

user=admin&pass=1234
```

Feature	GET	POST
Purpose	Retrieve data	Submit data / change server state
Data location	URL query string	Request body
Visible in address bar	Yes	No
Stored in history / bookmarks	Yes	No
Logged in server and proxy URL logs	Yes, with the values	URL only, not the body
Length limit	Limited by URL length	No practical limit
Idempotent / cacheable	Yes	No

2. Analyze which method is more suitable for login and why

POST is the suitable method. A login request sends credentials and creates a session, which changes server state, and POST is designed for this. It keeps the password out of the URL, so it is not shown on screen, saved in browser history, bookmarked, or copied when a link is shared. GET is meant for safe, repeatable requests such as searching or fetching pages.

POST alone does not encrypt the data. The credentials are still readable in transit unless the connection uses HTTPS, which is why the table rates POST only as "Moderate".

3. Identify security risks in using GET for authentication

- **Browser history and bookmarks:** the password is stored in plain text in the URL.
- **Server, proxy and CDN logs:** URLs are routinely logged, so credentials end up in log files that many people can read.
- **Referer header leakage:** when the user clicks an external link, the full URL may be sent to the other site.

- **Shoulder surfing:** the password is visible on the screen in the address bar.
- **Caching and sharing:** GET responses and URLs can be cached or shared by mistake.
- **CSRF and replay:** a login or action can be triggered just by opening a link, and the same URL can be replayed.

4. Suggest best practices for secure login implementation

1. **Use POST over HTTPS:** encrypt all traffic with TLS and redirect HTTP to HTTPS (HSTS).
2. **Hash passwords:** store them using Argon2id or bcrypt with a unique salt, never as plain text.
3. **Validate on the server:** do not rely only on browser checks.
4. **Secure session cookies:** set Secure, HttpOnly and SameSite attributes.
5. **Protect against brute force:** apply rate limiting, account lockout and CAPTCHA.
6. **Use CSRF tokens:** protect the login and other state-changing forms.
7. **Generic error messages:** show "Invalid username or password" so valid accounts are not revealed.
8. **Multi-factor authentication:** add a second factor for stronger protection.

Conclusion: GET exposes credentials in many places and must never be used for login. POST combined with HTTPS and the practices above forms the minimum secure approach for authentication.

Question 4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions: 1. Identify missing closing tags. 2. Analyze issues with the <img> tag. 3. Determine how browsers handle these errors. 4. Suggest a corrected version.

SOLUTION:

1. Identify missing closing tags

- **</h1>** is missing after "Welcome".
- **</p>** is missing after "This is a paragraph".

Other issues: the <html> tag has no lang attribute, and <head> has no <meta charset="UTF-8"> or viewport tag. The <img> tag does not need a closing tag in HTML5 because it is a void element.

2. Analyze issues with the <img> tag

- **Missing alt attribute:** alt is required by the HTML standard. Without it, screen readers cannot describe the image, and nothing is shown if the image fails to load.
- **Missing width and height:** the browser does not know the image size in advance, so the layout can shift when the image loads.
- **Not self-closed for XHTML:** valid in HTML5 as written, but in XHTML or XML-based documents it must be written as .

- **Placement:** because the is not closed, the image ends up inside the unclosed elements instead of being a separate block.

3. Determine how browsers will handle these errors

Browsers do not stop at errors. The parser repairs them using fixed recovery rules, and a start tag does not automatically close an open <h1>. The result is that the paragraph is nested inside the heading, and the image inside the paragraph:

```
body
 └── h1
      ├── "Welcome"
      └── p
           ├── "This is a paragraph"
           └── img
```

- The open elements are only closed automatically when </body> is reached.
- The paragraph text and image are displayed in the large, bold h1 style instead of normal body text.
- The page outline is wrong: assistive technologies read the whole paragraph as part of the main heading.
- The repair is consistent in modern browsers, but relying on it is poor practice, and XHTML or XML parsers would reject the document completely.

4. Suggest a corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

Conclusion: The missing </h1> and tags cause the paragraph and image to be nested inside the heading, producing wrong styling and structure. Closing all elements and adding alt, width and height to the image gives clean, accessible and standards-compliant HTML.

Question 5: Browser-Server Communication Flow

The following sequence describes a web request: (1) User enters a URL in the browser, (2) DNS lookup occurs, (3) HTTP request is sent, (4) Server processes request, (5) Response is returned, (6) Browser renders page.

Questions: 1. Interpret each step. 2. Identify where delays can occur. 3. Analyze the role of DNS. 4. Suggest optimizations.

SOLUTION:

1. Interpret each step in the communication flow

No.	Step	Interpretation
1	User enters a URL	The browser parses the URL into scheme (https), host (www.example.com), path and query. It first checks its own cache and then sends the host name for DNS lookup.

No.	Step	Interpretation
2	DNS lookup	The domain name is translated into an IP address, for example www.example.com to 93.184.216.34.
3	HTTP request is sent	The browser opens a TCP connection to the IP address (and completes a TLS handshake for HTTPS), then sends a request such as GET /index.html HTTP/1.1 with headers like Host, User-Agent and Accept.
4	Server processes request	The web server routes the request, runs application logic, reads files or queries a database, and prepares the content.
5	Response is returned	The server sends a status line (for example 200 OK), headers (Content-Type, Content-Length, Cache-Control) and the body, which is usually HTML.
6	Browser renders page	The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree, calculates layout and paints the page. Further requests are made for CSS, JavaScript, images and fonts.

Note: the given sequence does not list the TCP and TLS connection set-up. These happen between steps 2 and 3 and also add time.

2. Identify where delays can occur

Stage	Possible cause of delay
DNS lookup	Slow or distant DNS resolver, no cached record, long chain of lookups through root, TLD and authoritative servers.
Connection set-up	TCP three-way handshake and TLS negotiation, especially on high-latency networks.
Network transfer	Low bandwidth, packet loss, long physical distance to the server, large uncompressed files.
Server processing	Overloaded server, slow database queries, heavy server-side code, no caching.
Rendering	Render-blocking CSS and JavaScript, large images, many separate resource requests, heavy scripts.

3. Analyze the role of DNS in the process

DNS (Domain Name System) works like the internet's phone book. Browsers and servers communicate using IP addresses, but people use readable names, so DNS converts the domain name into the IP address. No request can be sent until this lookup completes, which makes DNS the first step on the critical path.

The lookup checks the browser cache, the operating system cache and then the recursive resolver of the ISP. If no cached answer exists, the resolver queries a root server, then the TLD server (.com), then the authoritative name server of the domain, and caches the answer for the duration of its TTL. A cached result takes only a few milliseconds, while a full lookup can take tens to hundreds of milliseconds. If DNS fails, the page cannot load at all even though the web server itself is working.

4. Suggest optimizations to improve performance

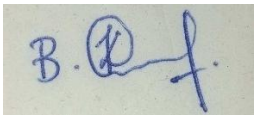
1. **Faster DNS:** use a fast resolver, set suitable TTL values, and use <link rel="dns-prefetch"> and preconnect for important hosts.
2. **CDN:** serve content from servers close to the user to reduce latency.
3. **Modern protocols:** use HTTP/2 or HTTP/3 and keep-alive connections to reuse connections and send requests in parallel.
4. **Compression:** enable gzip or Brotli for text resources.

5. **Caching:** use Cache-Control and ETag headers for browser and server-side caching.
6. **Reduce file size:** minify HTML, CSS and JavaScript, and use optimized formats such as WebP for images.
7. **Efficient rendering:** place CSS in the head, load scripts with defer or async, and lazy-load images below the fold.
8. **Server tuning:** optimize database queries, use application caching and scale the server when needed.

Conclusion: Loading a web page involves DNS resolution, connection set-up, request processing, transfer and rendering, and a delay at any stage slows the whole page. DNS is the first dependency, so caching it, together with CDNs, compression and caching, gives the biggest overall improvement.

Code of Conduct:

I B KIRTHANAA 192324244 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **B KIRTHANAA**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand